Computer Networks 275 (2026) 111954

Contents lists available at ScienceDirect

Computer Networks

journal homepage: www.elsevier.com/locate/comnet

From packets to predictions on GPU: Accelerated graph-based intrusion detection system

Ahmed Salah Tawfik Ibrahim [a], Emilio Paolini [b,*], Filippo Cugini [a], Francesco Paolucci [a]

[a] PNTLab CNIT, Pisa, Italy
[b] TeCIP Institute, Scuola Superiore Sant'Anna, Pisa, Italy

ARTICLE INFO

Keywords:
Graph neural networks (GNN)
GPU acceleration
Intrusion detection
Cybersecurity
CUDA

ABSTRACT

Graph Neural Networks (GNNs) are effective in detecting cyberattacks thanks to their ability to model network traffic, capturing structural relationships within the network. However, the high latency of the graph construction phase directly translates into higher overall prediction time, posing a challenge to real-time deployment. Therefore, an optimized GPU-accelerated framework that leverages the structural properties of the traffic graph is proposed in this work. It builds upon the notion of a precomputed adjacency matrix that gets modified by each graph instance. GPU threads are further used to construct the nodes, achieving an end-to-end graph generation and inference fully within the GPU. The sparsity of the graph and the coalesced memory access pattern within the GPU threads are further exploited to optimize the process. This allows the GPU to build large graphs much faster than the CPU without affecting the classification accuracy. This speedup is prominent for large graphs of 700 packets with the GPU being almost 4.3 times faster, highlighting the effectiveness of the proposed approach in real-time deployments.

1. Introduction

Graph Neural Networks (GNNs) have recently gained significant interest from both academia and industry thanks to their powerful capabilities in learning and representing non-Euclidean data [1]. By operating directly on graph-structured inputs, GNNs can model complex relational information that traditional Neural Network (NN) architectures often struggle to capture [2]. This property makes them a powerful tool for addressing network-related problems as they effectively capture complex dependencies and structures inherent to graph-based data [3,4]. In such settings, GNNs have demonstrated strong performance in tasks like traffic prediction, routing optimization, and anomaly detection [5,6].

As presented in [7], GNNs are particularly effective in intrusion detection, where modeling the relationships between entities (e.g., IP addresses, ports, and protocols) is crucial for accurate threat identification. To this end, a graph structure is particularly defined and constructed from raw packets to reflect the relationships between the different flows. In this context, the total prediction time $T_{Prediction}$, defined as the sum of the graph construction time T_{Graph} and the GNN inference time $T_{Inference}$ as shown in Eq. (1), emerges as a critical factor, especially for real-time or near-real-time detection scenarios.

$$T_{Prediction} = T_{Graph} + T_{Inference} \tag{1}$$

However, our analysis shows that $T_{Inference}$ is nearly negligible due to the lightweight architecture of the employed GNN model which requires minimal computational effort during the inference phase when executed on the GPU. As a result, the total prediction time is effectively dominated by the graph construction phase, i.e., $T_{Prediction} \approx T_{Graph}$. This observation is further supported by Fig. 1, where T_{Graph} demonstrates exponential growth as the classification window size increases, leading to a similar trend for $T_{Prediction}$. This underscores the importance of optimizing graph construction for real-time or near-real-time deployment of the intrusion detection system.

The primary reason behind this growth is that T_{Graph} is largely dominated by the construction of the adjacency matrix, a process that is both memory- and compute-intensive. Indeed, it has a computational complexity that scales quadratically with the number of nodes, making it particularly costly for large graphs. In practical scenarios, the number of nodes within a given classification window increases rapidly with the window size, which in turn exacerbates the growth of T_{Graph} and,

* Corresponding author.
E-mail addresses: ahmed.ibrahim@cnit.it (A.S. Tawfik Ibrahim), Emilio.Paolini@santannapisa.it (E. Paolini), filippo.cugini@cnit.it (F. Cugini), francesco.paolucci@cnit.it (F. Paolucci).

https://doi.org/10.1016/j.comnet.2025.111954
Received 23 September 2025; Received in revised form 27 November 2025; Accepted 16 December 2025
Available online 20 December 2025

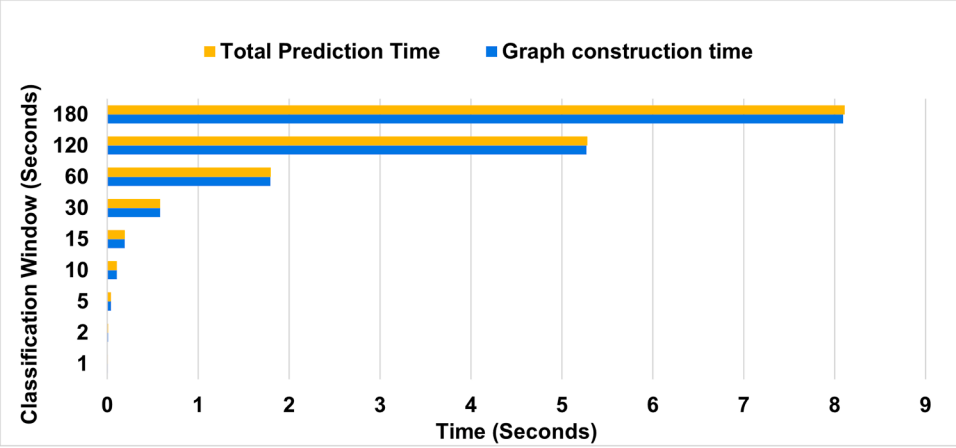

Fig. 1. Total prediction time vs graph construction time.

consequently, $T_{Prediction}$. This scalability issue introduces a significant bottleneck for the deployment and utilization of GNN-based systems in real-time environments.

Hence, to enable real-time deployment of GNN-based intrusion detection systems, it is essential to optimize the graph construction time, particularly the adjacency matrix generation. Previous work[7] introduced a CPU-based method to construct traffic graphs from raw packet data for GNN-based intrusion detection. That design emphasized algorithmic efficiency through choices such as limiting node granularity, constraining node degree, and tuning the classification window size (see details in Section 3), but these strategies inherently traded off scalability and responsiveness.

In this paper, a fundamental architectural shift is introduced: offloading not only GNN inference but also graph construction to the GPU. This enables an end-to-end in-GPU pipeline that directly addresses the scalability and latency bottlenecks that limit real-time deployment. Specifically, the node feature aggregation and adjacency matrix generation routines are redesigned to operate in parallel, fully leveraging the GPU's thread-level parallelism. This end-to-end execution within GPU memory not only reduces the computation time but also eliminates the overhead associated with CPU-GPU data transfers, enabling faster and more scalable intrusion detection.

Hence, as threats become more distributed and as networks grow in scale, flow-based graph structures face a performance challenge that this paper tries to address by accelerating the flow-based graph construction using the GPU, introducing a fast and scalable network security system.

2. Literature review

2.1. General review

Prior research has extensively investigated the use of GPUs in accelerating graph-related computations, highlighting their suitability for the highly parallel nature of such tasks.

In general, GPU programming follows the Single Instruction Multiple Data (SIMD) parallel programming paradigm [8], which allows a large number of threads to execute the same instruction simultaneously on different data elements. This model is particularly effective for data-parallel tasks that can be decomposed into independent units of work. Leveraging this paradigm, various solutions have been proposed to accelerate compute-intensive operations, such as sorting algorithms [9].

Serving the GPU threads, the GPU memory hierarchy offers a number of options to achieve memory access efficiency. In general, GPUs have many cores or Streaming Multiprocessors (SM) within which threads are executed. Threads are organized into blocks and each block is executed on one SM [10]. This leads to a memory hierarchy where each thread has access to its own local memory, its block's shared memory and the GPU's global memory. The sizes and speeds of these levels differ, with the global memory being the largest and slowest [11]. While parallelism can be thought of as executing all the threads in parallel, what really happens is that a number of threads, known as a warp, within each block, usually equal to 32, is what gets executed simulatenously. It is highly efficient when threads belonging to the same warp perform adjacent global memory accesses in what is known as memory coalescing [10]. Therefore, it is beneficial to consider these notions when designing algorithms to run on the GPU.

In the context of Deep Learning (DL), GPUs have played a pivotal role as key enablers, primarily due to their efficiency in accelerating matrix operations, which is an essential component of NN training and inference [12]. As graphs are commonly represented through matrices, such as the adjacency and feature matrices, GPUs offer a natural and efficient platform for accelerating graph-related computations [13].

Given a graph $G = (V, E)$, where V is a set of nodes and E is a set of edges, it can be represented by two matrices; the feature matrix X and the adjacency matrix A [14]. Each row of the feature matrix represents the features of one node in V, so it has a size of $|V|$ x n where n is the number of features. The adjacency matrix, however, is a square matrix of size $|V|$ x $|V|$ and its values are usually binary to indicate whether or not an edge exists between two nodes.

GNNs operate on these graphs to perform tasks such as node classification, edge prediction or graph classification [15]. These models typically follow a two-step process: (i) message passing and (ii) embedding update. During message passing, each node exchanges information with its neighbors by aggregating their feature representations. In the subsequent update step, the node combines the aggregated messages with its own embedding to produce a new representation. This aggregation can be implemented using various mechanisms, including Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs), which differ in how they weigh and combine information from neighboring nodes.

GNNs combine elements of DL and graph theory, which makes them especially well-suited for GPU acceleration. On one hand, GPUs can be used to speed up both the training and inference phases of GNN models, as discussed in [16]. Concerning the training phase, several methods have been proposed to reduce the size of the graph by performing sampling, sparsification, clustering or generation [16]. For inference, efforts focus on optimizing the GNN model itself, exploiting techniques such as pruning, quantization, and knowledge distillation. The survey also highlights various systems and specialized hardware platforms designed to further accelerate GNN execution.

On the other hand, GPU parallelism may also be exploited by leveraging the structural properties of graphs, as discussed in [17]. For

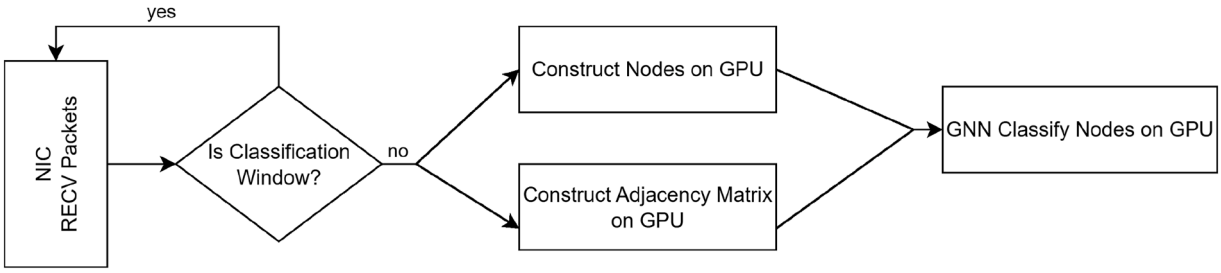

Fig. 2. Proposed system architecture.

instance, vertices can be distributed across GPU threads to enable parallel processing. However, ensuring consistency in such parallel environments often requires synchronization mechanisms. In message-passing frameworks like GNNs, where vertices exchange embeddings, it is essential to prevent race conditions using explicit synchronization barriers or atomic operations. Ultimately, the effectiveness of these acceleration techniques is highly application-dependent, as they must be tailored to the specific computational patterns and data access behavior of the task.

2.2. GPU graph structures

There exists a number of solutions implementing graph structures and algorithms within the GPU, depending on the nature of the graph. On one hand, static graphs are usually stored on GPUs in the form of a list of vertices and a sparse representation of the adjacency matrix. Dynamic graphs, on the other hand, are more challenging due to memory constraints, traditionally requiring a graph rebuild with each update. Therefore, some solutions have been proposed to overcome this issue.

For example, some solutions, like cuSTINGER [18] and hornet [19], support edge insertion and deletion while keeping the vertex set fixed. In doing so, CuSTINGER provides a flexible dynamic adjacency-list structure on GPUs and was an early demonstration of high-throughput GPU updates. However, its memory layout makes it inefficient for fine-grained, real-time insertions. Network telemetry typically arrives as many small, temporally ordered events rather than large, uniform batches, and CuSTINGER's reliance on batch-oriented processing leads to significant latency amplification and update stalls-undesirable for intrusion detection or online flow-tracking. Hornet, however, improves memory efficiency and update throughput through block-based adjacency expansion, but this design implicitly assumes bulk updates. In network environments, where packets arrive continuously and rapidly, Hornet's block resizing and metadata synchronization introduce additional overheads. These irregularities make it less suitable for maintaining network graphs where delay directly harms detection timeliness.

Other solutions, like FaimGraph [20], support both vertex and edge insertion and deletion. It does so by introducing page-based adjacency lists that remove global reallocation costs and are more resilient to dynamic updates. While this provides good stability for fluctuating graphs, the fine-grained paging model adds indirection and memory overhead that penalize fast updates. Vertex and edge insertions involve a number of checks and sorting steps, introducing an overhead that is not suitable for network-based graphs where vertices and edges are inserted rapidly due to the streaming nature of the problem.

In general, while these systems demonstrate impressive performance for general dynamic graph workloads, they do not directly address the constraints of network-derived dynamic graphs. Therefore, this paper proposes a hybrid approach where the benefits of a static structure is utilized to manage the memory requirements and the streaming nature of the intrusion detection problem, while the dynamic nature of a network graph is taken into account by applying modifications efficiently on the static data structure.

3. Scenario and methodology

To demonstrate the effectiveness of incorporating GPU acceleration, this paper considers the case of an intrusion detection system based on GNNs. This system transforms the packet features into a traffic graph. In the following sections, the system architecture along with a detailed description of the algorithms are presented.

3.1. Proposed architecture

The architecture of the proposed system is summarized in Fig. 2.

A classification window is defined within which packets are collected and parsed as they arrive where they get transfered to the GPU. There, the packets are organized into nodes whose features are computed by aggregating packet features. Furthermore, the adjacency matrix is computed to store the connectivity information of these nodes. Once the graph is ready, the nodes get classified, also on the GPU. This enables an end-to-end in-GPU intrusion detection.

In the following subsection, the implementation details of this architecture are clarified. Furthermore, upon inspecting the results in Section 4, the system is enhanced by applying a number of optimizations which are detailed in Section 5.

3.2. Methodology

Before describing the methodology, it is useful to recall some concepts and notations from [7]. A traffic graph $G = (V, E)$ is constructed from packets, where a node $v \in V$ represents a set of packets belonging to the same flow. This flow is identified by the IP addresses and ports of the source and destination endpoints along with the protocol as shown in Eq. (2).

$$flow(v) = (SrcIP, SrcPort, DstIP, DstPort, Protocol) \tag{2}$$

In addition, the source and destination of a node are defined as a tuple of the corresponding IPs, ports and the protocol as shown in Eqs. (3) and (4).

$$Src(v) = (SrcIP, SrcPort, Protocol) \tag{3}$$

$$Dst(v) = (DstIP, DstPort, Protocol) \tag{4}$$

The number of packets grouped in one node is referred to as the node granularity. This value acts as a graph hyperparameter, which was empirically optimized and set to 8 [7].

An edge e between two nodes v_1 and v_2 indicates one of two possible relationships:

- **R1:** the two nodes contain temporally consecutive packets belonging to the same flow; i.e., $flow(v_1) = flow(v_2)$ and $time(v_1) < time(v_2)$ and there exists no v such that $time(v_1) < time(v) < time(v_2)$;
- **R2:** the two packets belong to different flows but they are related. Two packets are related if the source of the first equals the destination of the second or vice versa; i.e., $flow(v_1) \neq flow(v_2)$ and $Src(v_1) = Dst(v_2)$ or $Src(v_2) = Dst(v_1)$.

In order to limit the number of edges in the graph, another hyperparameter, the maximum node degree, was experimentally determined to be equal to 50 [7] on the 5G-NIDD dataset [21]. Together with the node granularity, these two hyperparameters play a key role in preventing the graph from growing excessively in size.

Thus, graph construction involves the following steps: (i) collecting a window of packets; (ii) extracting features from those packets; (iii) grouping them into nodes; and (iv) building the corresponding adjacency matrix.

Building the nodes is a bucket-filling process where packets are considered chronologically as shown in Algorithm 1. The flow id of each packet is extracted and used as the node id. If the node is empty or if the node granularity has been reached, a new node is created where the packet is then inserted. Otherwise, a packet is inserted into the existing node where its features get aggregated with those of the inserted packets in that node. So, the complexity of this step, ignoring the complexity of the aggregation, is linear in the number of packets, i.e., $O(|P|)$.

Algorithm 1: Build nodes from packets.

Input : List of packets P, Node Granularity $NodeGranularity$
Output: List of Nodes V
$V \leftarrow [\,]$;
for $i \leftarrow 0$ **to** $|P|$ **do**
  $packet \leftarrow P[i]$;
  $FlowID \leftarrow flow(packet)$ **if**
   $0 < |V[FlowID]| < NodeGranularity$ **then**
    $V[FlowID] \leftarrow aggr(V[FlowID], packet)$;
  else
    $V[FlowID] \leftarrow packet$;

The next step involves adding edges to the constructed nodes; more formally, this corresponds to building the adjacency matrix. Nodes are processed in a chronological order: each node is compared with all subsequent nodes to determine whether any of the previously defined edge conditions are satisfied, as outlined in Algorithm 2. If a condition is met and neither node exceeds the maximum node degree constraint, an undirected edge is added between them. It can be noted then that the complexity of this step is quadratic in the number of nodes, or $O(|V|^2)$.

Algorithm 2: Build adjacency matrix from nodes.

Input : List of nodes V, maximum node degree $MaxNodeDegree$
Output: Adjacency matrix Adj
$Adj \leftarrow [\,]$;
for $i \leftarrow 0$ **to** $|V| - 1$ **do**
  $v_1 \leftarrow V[i]$;
  $Adj[i,i] \leftarrow 1$;
  for $j \leftarrow i+1$ **to** $|V|$ **do**
    $v_2 \leftarrow V[j]$;
    if $flow(v_1) = flow(v_2)$ ***and*** $consecutive(v_1, v_2)$ **then**
      $Adj[i,j] \leftarrow 1$;
      $Adj[j,i] \leftarrow 1$;
    else if $NumEdges(v_1) < MaxNodeDegree$ ***and***
     $NumEdges(v_2) < MaxNodeDegree$ **then**
      if $Src(v_1) = Dst(v_2)$ ***or*** $Src(v_2) = Dst(v_1)$ **then**
        $Adj[i,j] \leftarrow 1$;
        $Adj[j,i] \leftarrow 1$;

After the construction phase, in which the adjacency and feature matrices are built, both matrices are then transferred to the GPU to perform the inference using the GNN. Graph construction would be significantly faster if performed directly on GPU, as this would eliminate CPU-GPU transfer overhead and enable parallelization across GPU threads, leading to substantial time savings.

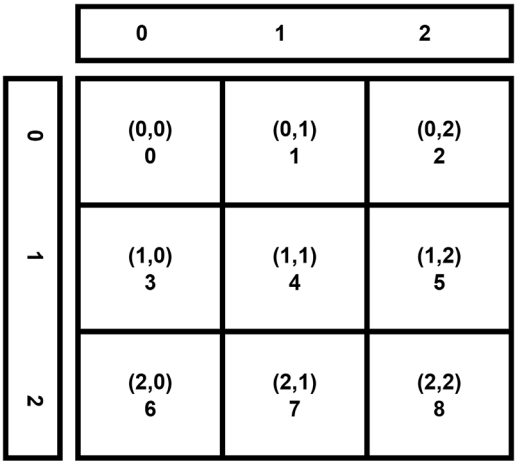

Fig. 3. Cartesian product of addresses.

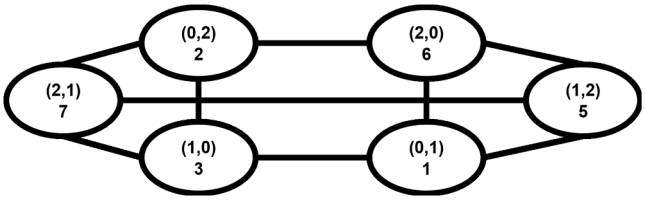

Fig. 4. Example Graph with 3 Addresses. This graph represents the scenario in which all the possible relationships (R1 or R2) among nodes are present.

The main challenge with this approach is that GPU programming typically requires memory to be pre-allocated; therefore the number of nodes must be known in advance.

Thus, the key idea is that if such number of nodes can be determined in advance, then the adjacency matrix of this graph can be built a-priori assuming there is a unique mapping between the matrix indices and the Src and Dst addresses defined in Eqs. (3) and (4).

In order to clarify this idea, consider a set of 3 integer addresses $Addr = \{0, 1, 2\}$. Out of the cartesian product of this set with itself AxA, a total number of 9 pairs can be obtained, which in essence could correspond to 9 flow ids, or 9 nodes in the graph as shown in Fig. 3.

Following the edge rule creation (see R1 or R2), the adjacency matrix corresponding to all the possible combinations of addresses can be built even before receiving any packets.

For instance, it can be observed that node 3 corresponding to the flow id (1, 0) will have edges with nodes 1, 2 and 7 as shown in the complete graph in Fig. 4.

Obviously, there are some dummy nodes, i.e., the ones containing the same address for source and destination; these will not end up in the final graph.

This process is deterministic if the number of addresses is known in advance. By leveraging one of the main findings of [7], a classification window of 75 packets is sufficient to make accurate predictions. This means that a classification session can contain a maximum of 150 addresses. Therefore, the cartesian product would lead to having 150^2 nodes which eventually results in an adjancency matrix $\in \{0,1\}^{150^2 \times 150^2}$.

Building this adjacency matrix is needed only once, so it can be done using GPU threads as outlined in Algorithm 3. Basically, each thread takes care of one node, filling the row and column corresponding to this node in the matrix with 1 if edge conditions hold.

Algorithm 3: Thread to initialize default adjacency matrix.

Input : Adjacency Matrix Adj, Maximum Number of Node Configurations N
Output: Initialized Adjacency Matrix Adj

```
row ← ThreadIdx.x ;
col ← ThreadIdx.y ;
if row ≠ col and row < n and col < n then
    NodeIndex ← row * N + col
    Adj[NodeIndex, NodeIndex] ← 1 for i ← 0 to N do
        if col ≠ i then
            Adj[NodeIndex, col * N + i] ← 1
        if row ≠ i then
            Adj[NodeIndex, i * N + row] ← 1
```

The problem with this paradigm arises when constructing the nodes from actual packets.

In practice, a node is identified by the flow id that is mapped to an integer. This integer maps to one and only one row in the matrix. This means that two nodes sharing the same flow id cannot be allowed. Therefore, an infinite node granularity and an infinite node degree are a must to achieve this unique mapping. It is worth recalling that the maximum values of these two hyperparameters were introduced to limit the graph size. However, in a GPU setting, the speedup achieved by the hardware acceleration is sufficient to discard these constraints.

At this point, the final chellenging step is mapping the actual source and destination addresses as defined in Eqs. (3) and (4) to integers. This mapping has to be bijective within the session at hand; meaning that an integer produced by this mapping has to correspond to one and only one address and each address has to correspond to one and only one integer. In case of 150 addresses, the integer set $N = \{0, 1, 2, \dots, 149\}$ is the output of this mapping. Thus, the goal is to learn a bijective function $f : x \mapsto N \; where \; x \in \{Src(v), Dst(v) \; \forall v \in V\}$. In this case, the most straightforward mapping technique would be a dictionary because it is relatively fast and does not lead to conflicts, ensuring the bijective property of the function. However, the implementation proposed in this paper is based on python that does not allow passing dictionary to the GPU. For such a reason, a k-means clustering algorithm[22] with k equal to the number of addresses is exploited. Simply, the addresses within a given session are represented as vectors that get mapped to integer cluster labels ranging between 0 and the number of addresses within the session. This ensures that each unique address will map to one and only one cluster label within the current session, guaranteeing the desired bijective property of the mapping function within the session. It is also worth noting that this mapping is not required nor expected to persist across different sessions because each session is treated independently. Note that the computational burden brought by k-means running on GPU is negligible. Once the initial graph and the mapping are built, the next step is to adjust them according to the actual observed flows. Therefore, a final adjusting step is needed where absent nodes are removed along with their corresponding edges (see Algorithm 5). This can also be done in parallel using the GPU threads, potentially speeding up the process. All the procedures are outlined in Algorithms 4–6.

Algorithm 4: Building the nodes using the GPU.

Input : Packet List P, Mapping f
Output: Session Graph $G(V, Adj)$

```
Idx ← ThreadIdx packet ← P[Idx] ;
SrcAddr ← Src(packet) ;
DstAddr ← Dst(packet) ;
NodeIndex ← f(SrcAddr) * |f| + f(DstAddr)
V[NodeIndex] ← AtomicAggr(V[NodeIndex], packet)
```

Algorithm 5: Adjusting the adjacency matrix using the GPU.

Input : Adjacency Matrix Adj, Nodes V
Output: Adjusted Adjacency Matrix Adj

```
Idx ← ThreadIdx if V[Idx] = ϕ then
    Adj[Idx, _] ← 0 Adj[_, Idx] ← 0
```

Algorithm 6: Building the graph using GPU.

Input : Packet List P, Initialized Adjacency Adj
Output: Session Graph $G(V, Adj)$

```
AddrSet ← Src(p) ∪ Dst(p) ∀p ∈ P ;
f ← GPU_kmeans(AddrSet, |P| * 2)
V ← GPUThreads_BuildNodes(P, f, V) ;
Adj ← GPUThreads_FixAdj(Adj, V) ;
```

It is trivial to assume that the graph construction time is the sum of the time intervals needed to complete each of the previously outlined steps; however, it is crucial to highlight that the actual total time is slightly different. In practice, the initialization step is not supposed to be executed for each graph. Since the initial values are the same for all matrices of the same size, it is sufficient to compute it once and store it. Hence, the total graph construction time T_{Graph} can be defined as the sum of the node construction time T_{Node} and the adjacency matrix modification time T_{Adjust} as shown in Eq. 5

$$T_{Graph} = T_{Node} + T_{Adjust} \tag{5}$$

Importantly, GPU acceleration shows an improvement during the construction of many subsequent graphs. In this case, the total time required to construct n graphs is computed as follows:

$$T_{Total} = T_{Init} + nT_{Graph} \tag{6}$$

4. Baseline results

To evaluate the proposed methodology, experiments were mainly conducted on the 5G-NIDD dataset to quantify the performance gains. These tests were executed on an ubuntu server equipped with an NVIDIA Tesla T4 GPU.[1] The server's CPU is an Intel(R) Xeon(R) Gold 6244 CPU @ 3.60GHz with 8 cores.

4.1. Graph initialization time vs window size

This test aims to measure the time required to initialize the graph following the procedure outlined in Algorithm 3 considering different classification windows, or more precisely, the parameter N in the algorithm, i.e., the cardinality of the address space within a specific classification window measured in packets. In this test, and due to the limitations of the GPU, classification windows of up to 75 packets are considered. These correspond to address spaces of a maximum cardinality N of up to 150 because $N = 2 * WindowSize$. The test was run 30 times on a dedicated GPU where the measurements were used to run a t-score test to ensure statistical significance. The average of these runs is reported in Fig. 5 along with the 95 % confidence interval.

The results in Fig. 5 show that the graph initialization time grows non-linearly with the classification window size. While initialization remains below 2ms for windows up to 40 packets, the time increases sharply for larger windows, reaching nearly ≈ 20ms at 75 packets. This highlights the quadratic complexity of the initialization step with respect to the address space size and emphasizes the need for further optimizations when targeting larger windows

[1] https://www.nvidia.com/en-us/data-center/tesla-t4/

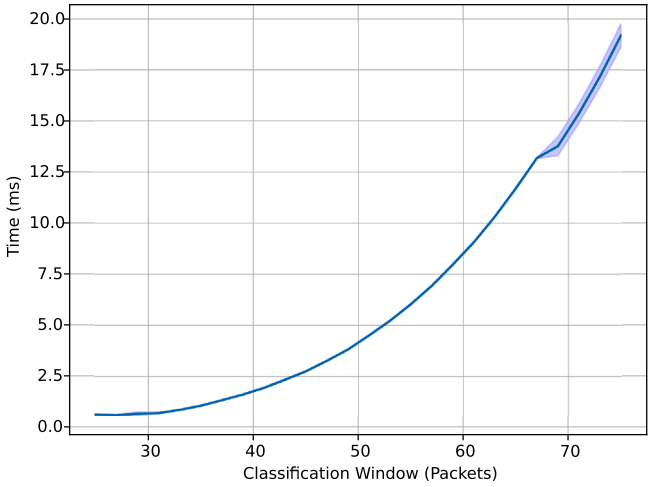

Fig. 5. Graph initialization time.

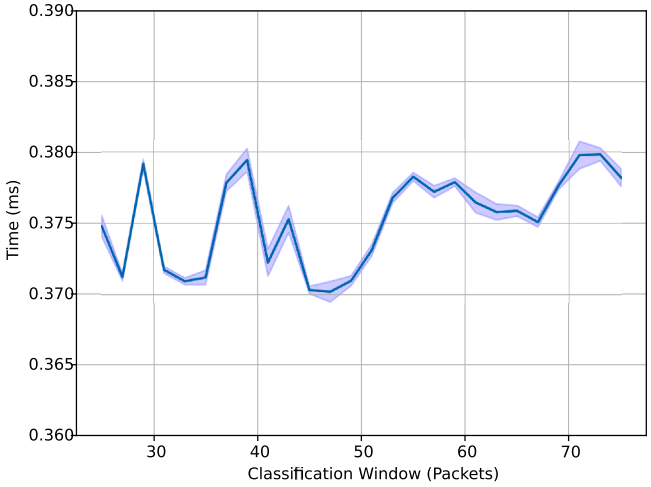

Fig. 7. Modifying the initial adjacency matrix.

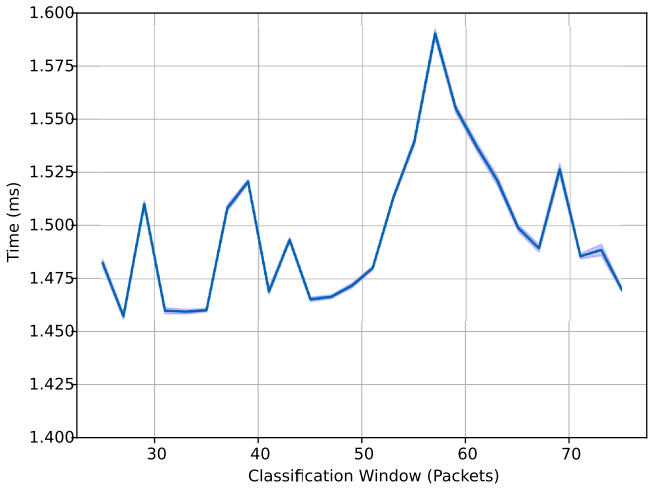

Fig. 6. Nodes construction time.

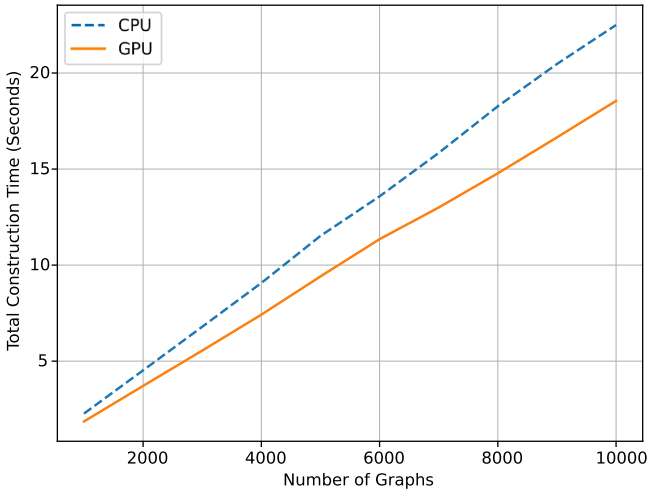

Fig. 8. CPU vs GPU graph construction time.

4.2. Time needed for building the nodes

In order to check the time needed to build the nodes, the packets in 5G-NIDD dataset are ordered by the time of arrival. Then, a subset of the packets is randomly selected to run the test. This experiment aims at considering different classification windows similar to Section 4.1. So, for each classification window, 10,000 graphs are constructed from the dataset following the procedure outlined in Algorithm 4. The average of the construction times of these graphs is considered as the required time to build a graph of that size using the GPU. In addition, a Z-score test is performed on those measurements to compute the 95 % confidence interval. The results of this experiment are shown in Fig. 6. In general, the time required remains almost constant in the range between 1.45 and 1.58 ms with 95 % confidence.

4.3. Time needed for adjusting the adjacency matrix

This experiment focuses on the last step of the process outlined in Section 3, namely adjusting the initial default adjacency matrix to the actual graph at hand. This step, unlike the initialization phase, is executed for every graph, making its quantification essential. The procedure follows the same methodology described in Section 4.2. Fig. 7 shows that the adjustment time remains consistently low, between 370 and 380 μs with 95 % confidence, even as the classification window size increases.

4.4. CPU Vs GPU

This experiment quantifies the performance gains achieved through the proposed GPU acceleration. The total construction time is measured for a varying number of graphs. Graphs are constructed considering a window size of 75 packets and a minimum number of 50 flow ids within this window. The graphs are constructed on both the CPU and the GPU in order to compare the total construction time. As shown in Fig. 8, the GPU is about 1.22 times faster than the CPU.

For the sake of completeness, the system is also evaluated by building and classifying 10,000 graphs using the GNN of[7]. The model consists of a 6-layer Graph Convolutional Network (GCN), followed by a fully connected layer with dropout for classification. The first three convolutional layers use 32 units each, while the last three use 8 units. This configuration was selected through empirical testing, as six layers were found to be the minimum depth required for high accuracy. Training is carried out with the Adam optimizer (learning rate 0.005, weight decay 0.0005) for 30 epochs, retaining the model version that achieves the best F1 score on the evaluation set.

The GPU-accelerated version achieves an average F1 score of 83.78 %, which is nearly identical to the 83.9 % obtained with the CPU implementation. The slight difference of 0.12 percentage points is negligible and can be attributed to minor variations introduced by GPU parallelism and floating-point operations, rather than to a loss in model capability.

5. System optimization and final results

While the approach described in Section 3 improves performance, it is not without limitations. Firstly, it is memory-hungry as it requires N^4 integers to store the adjacency matrix and $(num\_features + 1)N^2$ floating point numbers to store the node features and the labels. Assuming 32-bit floats and integers are used, that translates into $4(num\_features + 1)N^2 + 4N^4$ bytes of memory. For $N = 150$, this is 2 GB of GPU memory. To make matters worse, doubling N would require 30 gigabytes of memory. Since most GPUs provide 16–32 GB of memory and the majority of stored values are zeros, a memory-efficient representation is essential.

Secondly, performing writes to the global memory of the GPU is the most costly operation. This is due to the GPU memory hierarchy outlined in Section 2. Basically, since each block of threads has access to a shared memory, or a cache, it is generally better to perform writes onto the cache and then synchronize with the global memory in the end. In addition, if threads within a warp access consecutive memory locations from the global memory, then performance is enhanced as this is equivalent to a single memory transaction (as compared to random accesses which lead to multiple transactions).

In the following subsections, the methodology of Section 3 is modified to account for these optimizations and show the results.

5.1. Sparsity

As previously noted, the precomputed adjacency matrix is highly sparse. However, it has some symmetric properties that make it easy to analytically know the number of nonzero elements. For a matrix of size N^2 x N^2, the number of rows containing nonzero elements is $N^2 - N$. This is because there are N rows that correspond to flow IDs where the source is the same as the destination, i.e., (0,0), (1,1), (2,2), . etc. These rows are all zeros because such interactions are of no interest to an intrusion detection system. For the remaining rows, the number of nonzero elements per row is $2N - 3$. Since we are only interested in the locations of the nonzero elements, the indices of the rows and columns of those elements can be stored instead of the complete adjacency matrix. This is known as the coordinate format (COO). In general, the nonzero values are also stored along with their locations; however, the nonzero values are all equal to 1 in undirected unweighted graphs like ours, so storing these values is not needed. This alternative representation significantly reduces the memory requirement as it goes from $4N^4$ bytes to $8(N^2 - N)(2N - 3)$ bytes. Fig. 9 showcases this gain in memory, highlighting the exploding property of storing the complete sparse adjacency matrix with respect to the COO representation.

Applying this optimization requires revising the proposed algorithm. In particular, the initialization and adjusting steps have to be modified. A trivial modification would be to precompute the default nonzero locations in the initialization phase and then modify them for each graph. However, experiments show that while the initialization step is significantly sped up, the adjusting step experiences a significant drop in performance. This is due to the fact that a very small subset of the nonzero elements is actually present in each graph instance, which leads to a huge number of global memory writes which worsens the performance.

In that case, it is better to perform a small number of writes which can be achieved by discarding the initialization step and performing writes only for nodes that actually exist in the graph instance. In other words, the initialization and adjustment steps are now merged into one step that builds the current graph's adjacency matrix in the COO represnetation.

Nonetheless, with the COO representation the direct correspondence between the thread indices and the adjacency matrix index no longer holds. However, another correspondence can be computed based on the thread index by performing a shift. If each thread is responsible for one node, then each thread is responsible for $2N - 3$ locations in the COO representation. But the total number of nodes is not N^2 nodes because the rows containing all zeros have been discarded. So, the start index to write to by a given thread cannot be computed by multiplying the node index of that thread by $2N - 3$. Instead, a factor has to be subtracted from the node index in order to get the correct start index. This factor is computed by dividing the thread index by $N + 1$. The whole process is shown in detail in Algorithm 7.

Algorithm 7: Revised adjacency building.

Input : Rows $Rows$, Columns $Cols$, Nodes V
Output: Nonzero Rows $Rows$, Nonzero Columns $Cols$

```
idx ← ThreadIdx.x idy ← ThreadIdx.y if idx < N and idy < N
 and idx ≠ idy then
    Node1 ← idx * n + idy Shift ← Node1/(N+1)
    NNZ_Idx ← (Node1 − Shift) * (2N − 3)
    ExistsNode1 ← Nodes[Node1] if ExistsNode1 then
        count ← 0 for i ← 0 to N do
            if i ≠ idy then
                Node2 = idy * N + i
                ExistsNode2 = Nodes[Node2] if
                ExistsNode2 then
                    Rows[NNZ_Idx + count] ← Node1
                    Cols[NNZ_Idx + count] ← Node2
                    count ← count + 1
        for i ← 0 to N do
            if i ≠ idy and i ≠ idx then
                Node2 = i * N + idx
                ExistsNode2 = Nodes[Node2] if
                ExistsNode2 then
                    Rows[NNZ_Idx + count] ← Node1
                    Cols[NNZ_Idx + count] ← Node2
                    count ← count + 1
```

In order to test this algorithm, the adjacency building time is measured by changing N. The results of this experiment are shown in Fig. 10.

As shown in Fig. 10, the optimized adjacency construction exhibits an overall increasing trend with the classification window size. The execution time remains very low for small windows, but grows as the number of packets increases, reflecting the higher complexity of handling larger address spaces. Small fluctuations are visible for larger windows (beyond 250 packets), which are attributed to GPU memory constraints and synchronization overheads.

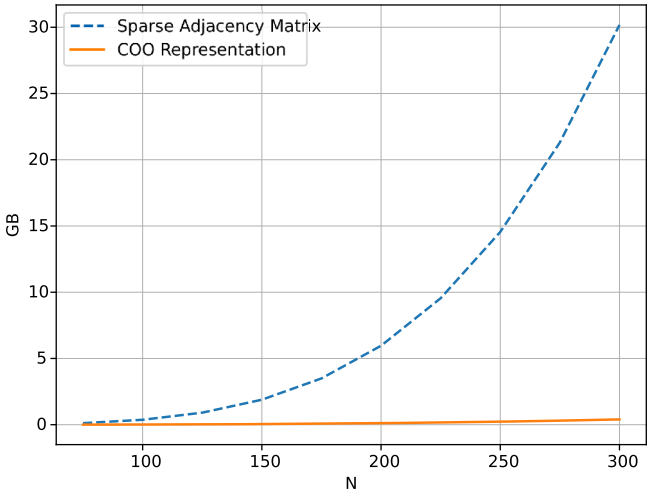

Fig. 9. Memory requirement per number of addresses (N).

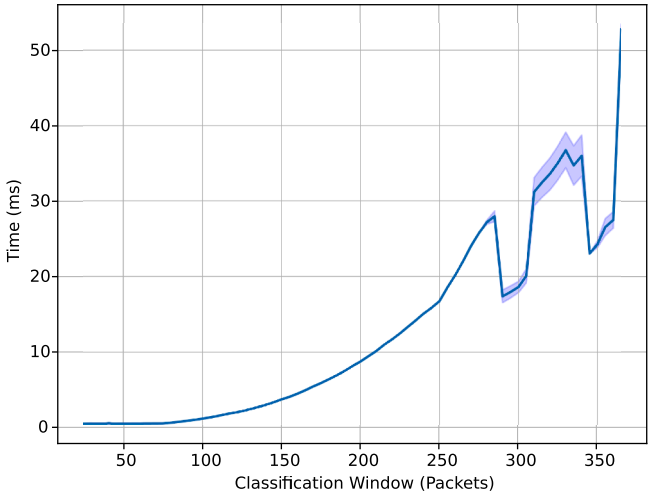

Fig. 10. Optimized adjacency construction.

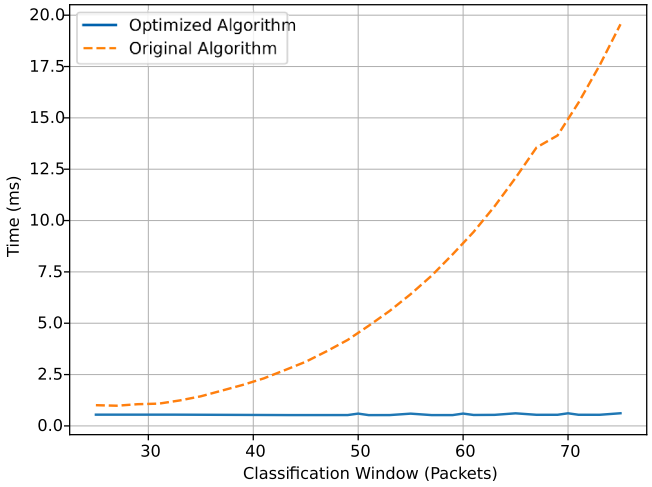

Fig. 11. Time required to build one graph.

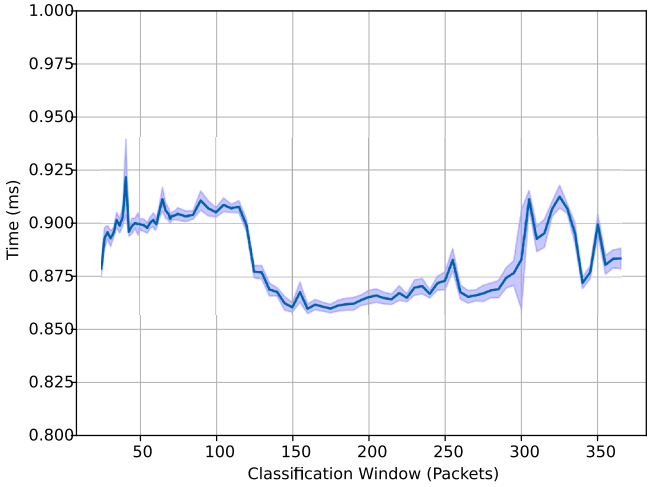

Fig. 12. Optimized node construction time.

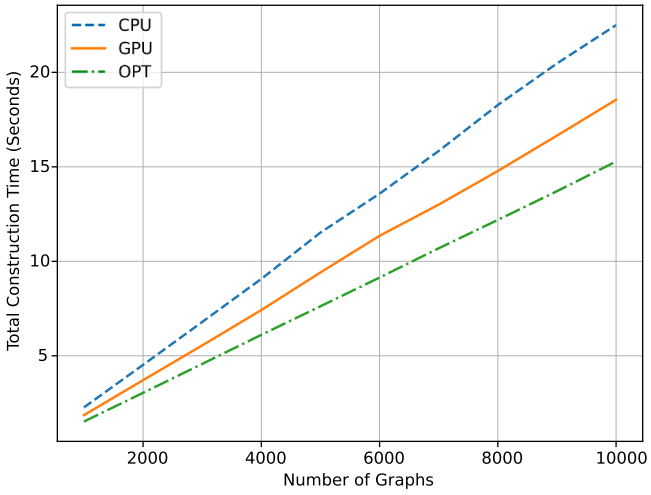

Fig. 13. CPU vs GPU vs optimized GPU.

As a matter of fact, the time needed to build one graph using the optimized algorithm is much less than the time needed by the original algorithm as shown in Fig. 11 where it remains almost the same regardless of the classification window in the optimized case.

5.2. Memory access

In order to achieve memory coalescing,[2] it is important to consider the memory access patterns of the algorithms.

In the optimized adjacency building algorithm, memory coalescing is achieved because accesses to the rows and columns memory locations are consecutive with respect to the threads accessing them. Normally, the conditions included within the algorithm would result in poor performance due to synchronization barriers; however, the fact that they significantly limit the number of writes to global memory results in an enhanced performance. So, this algorithm does not require further optimizations when it comes to its memory access patterns.

As for the node building algorithm, the writes to global memory are not coalesced because the calculation of the node index does not depend on the thread index. In fact, consecutive threads may access very far locations in global memory since these accesses are governed by the node index and not the thread index. A better way to optimize this algorithm is by using the shared memory. The shared memory is used to perform all the required computations and then write to global memory in the end. This is described in Algorithm 8.

Algorithm 8: Optimized node building.

Input : Packet List P, Mapping f
Output: Session Graph $G(V, Adj)$
$Idx \leftarrow ThreadIdx$
$SharedMemIn \leftarrow P$
$SharedMemOut \leftarrow []$
$packet \leftarrow SharedMem[Idx]$
$SrcAddr \leftarrow Src(packet)$
$DstAddr \leftarrow Dst(packet)$
$NodeIndex \leftarrow f(SrcAddr) * |f| + f(DstAddr)$
$SharedMemOut[NodeIndex] \leftarrow$
  $AtomicAggr(SharedMemOut[NodeIndex], packet)$
$Synchronize()$
$V \leftarrow SharedMemOut$

Testing this algorithm yields an improvement compared to the original algorithm. Fig. 12 shows the time required to build the nodes as a function of the classification window size. It is almost around 0.9 milliseconds with 95 % confidence.

[2] https://en.wikipedia.org/wiki/Coalescing_(computer_science)

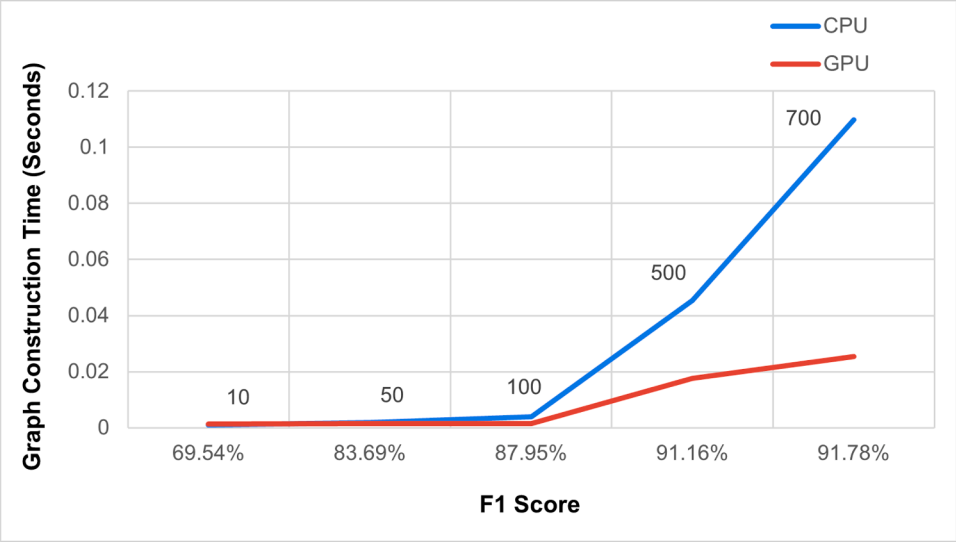

Fig. 14. CPU vs GPU - F1-time curve for different classification windows.

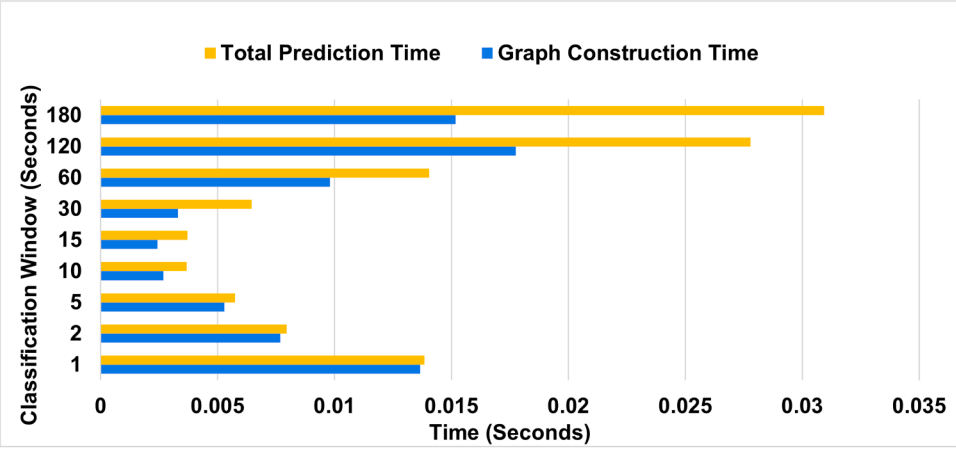

Fig. 15. Graph construction time vs total prediction time after optimization.

Table 1
Performance on different datasets.

Dataset	Window Size	Latency (ms)	F1 Score(%)
CIC-DDoS2019	10	1.4	80.90
	50	1.5	81.97
	100	1.5	81.99
	500	l7.2	78.68
	700	27	82.73
5G-NIDD	10	l.4	69.54
	50	l.5	83.69
	100	l.5	87.95
	500	l7.6	91.16
	700	25.4	91.78

Finally, to compare the optimized algorithm against the CPU and the original GPU algorithms, the time required by each of them to build a varying number of graphs has been measured. The graphs have a fixed classification window of 75 packets. The optimized one is on average 1.48× faster than the CPU and 1.21× faster than the original GPU algorithm as shown in Fig. 13. This might seem like an insignificant improvement; however, the real power of GPU parallelism appears with larger graphs; i.e., larger window sizes.

In fact, the limited classification window of 75 packets has the sole purpose of enabling real-time deployment on the CPU. With the GPU, this constraint is not needed because the optimized GPU acceleration has the power to achieve real-time classification with larger window sizes. This has the added benefit of enhancing the classification accuracy because larger graphs potentially lead to better classification. By measuring the graph construction time and f1 scores of window sizes of 10, 50, 100, 500 and 700 packets, Fig. 14 shows that with larger window sizes, the f1 score is better. It further shows that the GPU is capable of achieving an f1 score of 91 % in almost half the time of the CPU thanks to its ability of constructing graphs of larger window sizes much faster. This speedup is even more prominent for a classification window of 700 where the CPU requires 110 millseconds to build the graph while the GPU does it in only 25 milliseconds, almost quarter the time.

In fact, the efficacy of the system has also been verified by running it on the CIC-DDoS2019 dataset [23], highlighting similar performance in terms of latency as shown in Table 1. When it comes to the f1 score, it is slightly worse than 5G-NIDD. This difference in the prediction accuracy is attributed to the large size of CIC-DDoS2019 with respect to 5G-NIDD, the dataset that has been used to train the GNN originally. In fact, CIC-DDos2019 contains attack types that the GNN has not been trained on. This drop in the accuracy, however, is not dramatic, highlighting the efficacy of the system.

This is further reinforced by Fig. 15 which shows the ability of the GPU to construct much larger graphs much faster than the CPU times shown in Fig. 1. In fact, the maximum total time is reduced from 8 s on the CPU to almost 30 milliseconds on the GPU.

6. Conclusion

Graph construction time remains the dominant bottleneck in GNN-based intrusion detection, preventing efficient real-time deployment. To overcome this challenge, a GPU-accelerated system that offloads both graph construction and inference to the GPU is introduced. The design exploits thread-level parallelism and a precomputed adjacency matrix, which is adjusted per graph instead of being rebuilt from scratch. This architectural shift achieves a 1.22× speedup compared to the CPU baseline.

To address the memory overhead of storing large, sparse adjacency matrices, the system is further optimized using a COO sparse representation, memory coalescing, and shared memory operations. Respecting the 75 packet classification window constraint, these enhancements lead to a 1.48× speedup over the CPU implementation.

The real power of the GPU is further shown when this constraint is removed, unlocking the ability to build larger graphs which lead to a better classification accuracy in a much lower time. In fact, with a classification window of 700 packets, the GPU is almost 4.5 times faster than the CPU, with a higher f1 score than the 75 packet case.

The proposed solution opens the door for offloading GNN-based security function on SmartNICs equipped with GPUs, showing yet another benefit of this solution in computer networks and security.

Achieving near real-time classification with larger graphs does not mean that the GPU is free from limitations. For instance, the size of the largest graph that can be constructed is limited by the GPU memory size. Hence, future work will explore the use of reduced-precision number formats (e.g., Bfloat16) to lower execution time and memory footprint, allowing the construction of even larger graphs. Furthermore, different techniques to enhance the accuracy of the system, like data augmentation, will be applied. In addition, the GNN will be trained on larger datasets, encompassing more attack types to enhance the generalizability of the system. Finally, extending the framework to parallel multi-GPU settings could enable the simultaneous construction of multiple graphs, paving the way for distributed inference and improved scalability.

CRediT authorship contribution statement

Ahmed Salah Tawfik Ibrahim: Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization; **Emilio Paolini:** Writing – review & editing, Resources, Conceptualization; **Filippo Cugini:** Writing – review & editing, Supervision, Project administration, Funding acquisition; **Francesco Paolucci:** Writing – review & editing, Supervision.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the Chips Joint Undertaking (JU), European Union (EU) HORIZON-JU-IA, under grant agreement No. 101140087 (SMARTY), including top-up funding by the Italian MUR.

References

[1] Z. Zhu, F. Li, G. Li, Z. Liu, Z. Mo, Q. Hu, X. Liang, J. Cheng, Mega: a memory-efficient gnn accelerator exploiting degree-aware mixed-precision quantization, in: 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA), IEEE, 2024, pp. 124–138.
[2] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, P.S. Yu, A comprehensive survey on graph neural networks, IEEE Trans. Neural Netw. Learn. Syst. 32 (1) (2020) 4–24.
[3] S. He, Q. Luo, R. Du, L. Zhao, G. He, H. Fu, H. Li, STGC-GNNs: a GNN-based traffic prediction framework with a spatial–temporal Granger causality graph, Physica A 623 (2023) 128913.
[4] T.-L. Huoh, Y. Luo, P. Li, T. Zhang, Flow-based encrypted network traffic classification with graph neural networks, IEEE Trans. Netw. Serv. Manage. 20 (2) (2022) 1224–1237.
[5] Z. Wang, J. Hu, G. Min, Z. Zhao, Z. Chang, Z. Wang, Spatial-temporal cellular traffic prediction for 5G and beyond: a graph neural networks-based approach, IEEE Trans. Ind. Inf. 19 (4) (2022) 5722–5731.
[6] M. Ding, Y. Guo, Z. Huang, B. Lin, H. Luo, GROM: A generalized routing optimization method with graph neural network and deep reinforcement learning, J. Netw. Comput. Appl. 229 (2024) 103927.
[7] A.S.T. Ibrahim, E. Paolini, F. Cugini, F. Paolucci, Real-time graph neural network for malicious traffic detection, in: 2025 IEEE International Conference on Machine Learning for Communication and Networking (ICMLCN), 2025, pp. 1–6. https://doi.org/10.1109/ICMLCN64995.2025.11139379
[8] R. Baraglia, G. Capannini, F.M. Nardini, F. Silvestri, Sorting using bitonic network with CUDA, CEUR Workshop Proc. 480 (2009), pp. 33–40.
[9] D.I. Arkhipov, D. Wu, K. Li, A.C. Regan, Sorting with GPUs: a survey, CoRR abs/1709.02520 (2017). http://arxiv.org/abs/1709.02520. 1709.02520
[10] N. Fauzia, L.-N. Pouchet, P. Sadayappan, Characterizing and enhancing global memory data coalescing on GPUs, in: 2015 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), 2015, pp. 12–22. https://doi.org/10.1109/CGO.2015.7054183
[11] P. Hijma, S. Heldens, A. Sclocco, B. van Werkhoven, H.E. Bal, Optimization techniques for GPU programming, ACM Comput. Surv. 55 (11) (2023). https://doi.org/10.1145/3570638
[12] W. Jeon, G. Ko, J. Lee, H. Lee, D. Ha, W.W. Ro, Chapter six - deep learning with GPUs, in: S. Kim, G.C. Deka (Eds.), Hardware Accelerator Systems for Artificial Intelligence and Machine Learning, 122 of *Advances in Computers*, Elsevier, 2021, pp. 167–215. https://doi.org/10.1016/bs.adcom.2020.11.003
[13] D. Grinberg, An introduction to graph theory, (2025). https://arxiv.org/abs/2308.04512.
[14] J.H. Tanis, C. Giannella, A.V. Mariano, Introduction to graph neural networks: a starting point for machine learning engineers, (2024). https://arxiv.org/abs/2412.19419.
[15] W.L. Hamilton, Graph representation learning, Synth. Lectures Artif. Intell. Mach. Learn. 14 (3) (2025) 1–159.
[16] S. Zhang, A. Sohrabizadeh, C. Wan, Z. Huang, Z. Hu, Y. Wang, Yingyan, Lin, J. Cong, Y. Sun, A survey on graph neural network acceleration: algorithms, systems, and customized hardware, 2023, https://arxiv.org/abs/2306.14052.
[17] H. Wang, L. Geng, R. Lee, K. Hou, Y. Zhang, X. Zhang, SEP-graph: finding shortest execution paths for graph processing under a hybrid framework on GPU, in: Proceedings of the 24th Symposium on Principles and Practice of Parallel Programming, PPoPP '19, Association for Computing Machinery, New York, NY, USA, 2019, p. 38–52. https://doi.org/10.1145/3293883.3295733
[18] O. Green, D.A. Bader, CuSTINGER: supporting dynamic graph algorithms for GPUs, in: 2016 IEEE High Performance Extreme Computing Conference (HPEC), 2016, pp. 1–6. https://doi.org/10.1109/HPEC.2016.7761622
[19] F. Busato, O. Green, N. Bombieri, D.A. Bader, HorNet: an efficient data structure for dynamic sparse graphs and matrices on GPUs, in: 2018 IEEE High Performance Extreme Computing Conference (HPEC), 2018, pp. 1–7. https://doi.org/10.1109/HPEC.2018.8547541
[20] M. Winter, D. Mlakar, R. Zayer, H.-P. Seidel, M. Steinberger, FaimGraph: high performance management of fully-dynamic graphs under tight memory constraints on the GPU, in: SC18: International Conference for High Performance Computing, Networking, Storage and Analysis, 2018, pp. 754–766. https://doi.org/10.1109/SC.2018.00063
[21] S. Samarakoon, Y. Siriwardhana, P. Porambage, M. Liyanage, S. Y. Chang, J. Kim, J. Kim, M. Ylianttila, 5G-NIDD: a comprehensive network intrusion detection dataset generated over 5G wireless network, in: IEEE Dataport, 2022. https://doi.org/10.21227/xtep-hv36
[22] P. Vora, B. Oza, et al., A survey on k-mean clustering and particle swarm optimization, Int. J. Sci. Modern Eng. 1 (3) (2013) 24–26.
[23] I. Sharafaldin, A.H. Lashkari, S. Hakak, A.A. Ghorbani, Developing realistic distributed denial of service (DDos) attack dataset and taxonomy, in: 2019 International Carnahan Conference on Security Technology (ICCST), 2019, pp. 1–8. https://doi.org/10.1109/CCST.2019.8888419